

Robust Mesh Quality Enhancement and Adaptive Remeshing

CENG589 Digital Geometry Processing Term Project

Esad Mazi

June 25, 2026

Abstract

Triangle meshes produced by CAD tessellators or other automatic conversion tools may contain very skinny or nearly degenerate triangles. These triangles are visually small, but they are harmful for geometry processing because they lead to unstable normals, poor numerical conditioning, and unreliable downstream operations. This project implements a mesh-quality pipeline inspired by Botsch and Kobbelt’s robust degenerate-face removal procedure [1] and Donyach et al.’s adaptive isotropic remeshing approach [2].

The implemented system reads OFF meshes, measures triangle quality, visualizes bad triangles, applies conservative remeshing operations, and finally removes remaining degenerate triangles using targeted local operations. The pipeline was tested on the provided joint model and all four car models. On the joint model, the most aggressive pipeline reduced the number of detected bad triangles from 258 to 0 while preserving closed topology. A later follow-up experiment showed an important trade-off: the uniform remeshing stage improves quality metrics but can visibly deform the original shape, while a no-uniform variant preserves the shape better but leaves a small number of bad triangles. On the car models, targeted cleanup removed most detected bad triangles without introducing non-manifold edges.

1 Introduction

The initial motivation of this project was practical rather than theoretical: the provided joint input mesh visibly contained many thin triangles, while the provided joint output mesh appeared to be a cleaned reference result. The main goal was therefore to build a reproducible pipeline that can explain, measure, and improve this difference.

The project follows three principles:

1. Quality must be measured, not only inspected visually.
2. Geometry processing operations must preserve topology whenever possible.
3. Intermediate failures should be treated as observations, because they explain why more careful algorithms are necessary.

This resulted in a pipeline that is intentionally simple but measurable. It does not try to be a complete industrial mesh repair system; instead, it focuses on a clear term-project scope: detect problematic triangles, improve them using local remeshing operations, compare the result against the provided reference output, and document the behavior on additional models.

2 Background

Botsch and Kobbelt classify degenerate triangles into two important categories: needles and caps [1]. Needles have very small angles and are usually long and thin. Caps have an angle

close to 180.00° . The paper argues that these two cases should not be handled identically. In particular, needles can often be removed by collapsing short edges, while caps are more safely handled by splitting or slicing operations.

Dunyach et al. propose adaptive isotropic remeshing for interactive mesh deformation [2]. Their key idea is to use a target edge length that changes over the surface: high-curvature regions should be sampled more densely, while flatter regions can be represented with larger triangles.

This project uses these ideas in a practical form:

- triangle quality analysis and classification,
- conservative edge split and edge collapse operations,
- targeted cleanup for needles and caps,
- a curvature-based adaptive remeshing stage.

3 Quality and Topology Metrics

For each triangle, the implementation computes the minimum angle, maximum angle, shortest-edge / longest-edge aspect ratio, and near-zero area status. The thresholds used in the experiments are shown in Table 1.

Table 1: Bad triangle labels used in the experiments.

Label	Criterion
Needle-like	Minimum angle below 5.00° or aspect ratio below 0.05
Cap-like	Maximum angle above 175.00°
Near-zero area	Area below a scale-dependent epsilon

Topology is measured separately using boundary edge count, non-manifold edge count, and Euler characteristic. This separation became important during development. An early remeshing version improved some angle metrics but created holes and non-manifold edges. Therefore, the final pipeline reports both triangle quality and topology.

4 Implementation

The project is implemented in Python as a small command-line package named `meshfix`. The main modules are summarized in Table 2.

Table 2: Main implementation modules.

Module	Purpose
<code>meshfix.io</code>	OFF mesh reading and writing
<code>meshfix.quality</code>	Triangle quality metrics and bad triangle classification
<code>meshfix.visualize</code>	Colored PLY export and Polyscope visualization
<code>meshfix.topology</code>	Boundary and non-manifold edge analysis
<code>meshfix.remesh</code>	Uniform and adaptive remeshing operations
<code>meshfix.cleanup</code>	Targeted needle/cap cleanup
<code>meshfix.curvature</code>	Curvature proxy and adaptive sizing field

Bad triangles are visualized with a simple color scheme: gray for acceptable triangles, red for needle-like triangles, orange for low aspect ratio triangles, purple for cap-like triangles, and black for near-zero area triangles.

In addition to the Python implementation, the submission package includes a dependency-free C++17 port. This port contains the core OFF input/output, quality metrics, topology checks, and targeted degenerate-triangle cleanup. The Python version remains the main experimental implementation because it also contains visualization scripts, diagnostic figure generation, and the uniform/adaptive remeshing experiments. The C++ version is therefore used as a compiled reference implementation of the main measurable cleanup logic.

4.1 Algorithm Pseudocode

The implemented workflow is intentionally written as a sequence of measurable stages. The high-level algorithm is:

Algorithm 1: Mesh quality enhancement pipeline

Input : OFF mesh M

Output: cleaned mesh M_{final} , metrics, visual diagnostics

```

1  M0 <- read_off(M)
2  Q0 <- analyze_quality(M0)
3  T0 <- analyze_topology(M0)
4  export_quality_visualization(M0)

5  if full remeshing experiment is enabled then
6      Mu <- conservative_uniform_remesh(M0)
7      Qu <- analyze_quality(Mu)
8      Tu <- analyze_topology(Mu)
9  else
10     Mu <- M0

11 Mc <- targeted_degenerate_cleanup(Mu)
12 Qc <- analyze_quality(Mc)
13 Tc <- analyze_topology(Mc)

14 if adaptive remeshing experiment is enabled then
15     Ma <- curvature_adaptive_remesh(Mc)
16     M_final <- targeted_degenerate_cleanup(Ma)
17 else
18     M_final <- Mc

19 export OFF, CSV metrics, topology report, and colored PLY figures

```

The targeted cleanup stage is the most important local repair component:

Algorithm 2: Targeted degenerate cleanup

Input : mesh M , number of iterations K

Output: locally repaired mesh M

```

for iter = 1..K:
    bad_faces <- classify_all_faces_of(M)
    collapse_edges <- empty_set
    split_edges <- empty_set

    for each bad face f:
        if f is cap-like:

```

```

        add longest edge of f to split_edges
    else if f is needle-like or low-aspect:
        add shortest edge of f to collapse_edges

for each edge e in collapse_edges ordered by length:
    collapse e only if the link/topology and area checks pass

recompute bad faces after collapses
for each remaining cap-like face:
    split its longest edge consistently across all incident faces

remove duplicate and zero-area faces

```

For the car3 case study, individual face indices are not stable because edge collapse and split operations delete and create triangles. Therefore, local regions are tracked using the original bad-triangle centroid:

Algorithm 3: Car3 local bad-triangle tracking

Input : car3 mesh and cleanup steps M0...M10

Output: local patch figures and tracking CSV

```

1  select several severe bad faces from M0
2  store each selected face centroid as a seed point
3  for every cleanup step Mt:
4      find the face nearest to each seed point
5      extract a local patch around the seed point
6      classify all patch faces
7      record whether the tracked local face is bad or good
8      export colored patch PLY and CSV statistics

```

5 Method

5.1 Baseline Analysis

The first step is to read the input OFF mesh and compute quality statistics without changing the geometry. This gives an objective baseline for comparison. For example, `joint_input.off` contains 258 detected bad triangles, while the provided `joint_output.off` contains none under the same thresholds.

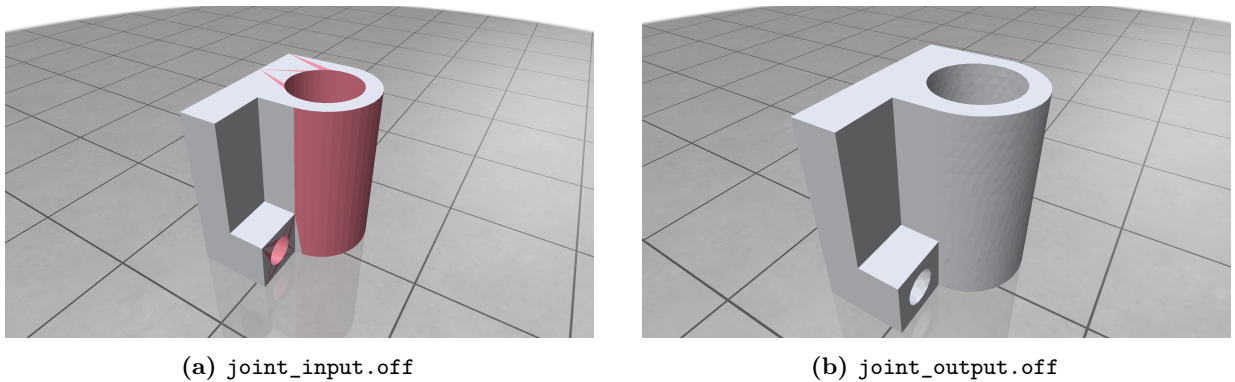


Figure 1: Baseline quality visualization of the provided joint meshes. Red triangles indicate needle-like degenerate triangles, while gray triangles pass the quality tests.

5.2 Conservative Uniform Remeshing

The first remeshing stage attempts to regularize edge lengths using local operations: split edges that are too long, collapse edges that are too short, and reject operations that would create invalid topology.

The initial implementation was intentionally tested visually and numerically. It revealed a common remeshing problem: splitting one triangle without consistently splitting its neighbor can create cracks or T-junction-like artifacts. This was fixed by globally consistent edge splitting, where all triangles sharing a split edge use the same midpoint.

Another issue appeared with aggressive smoothing and edge flips. They improved some angle statistics but caused visible deformation and small topology defects. For this reason, the final uniform remeshing baseline uses conservative split and collapse operations, while flip and smoothing are kept as optional experimental flags.

A separate visual inspection of this stage shows that even conservative uniform remeshing changes the original joint shape more than desired. This is not a topology failure: the mesh remains closed and non-manifold-free. The problem is geometric fidelity. Uniform remeshing is a global operation; once the coarse joint input is resampled into a much denser mesh, later targeted cleanup or adaptive remeshing works on the already modified surface and cannot recover the original shape exactly. Therefore, the uniform-based result is treated as a quality-optimized experiment rather than a perfect shape-preserving output.

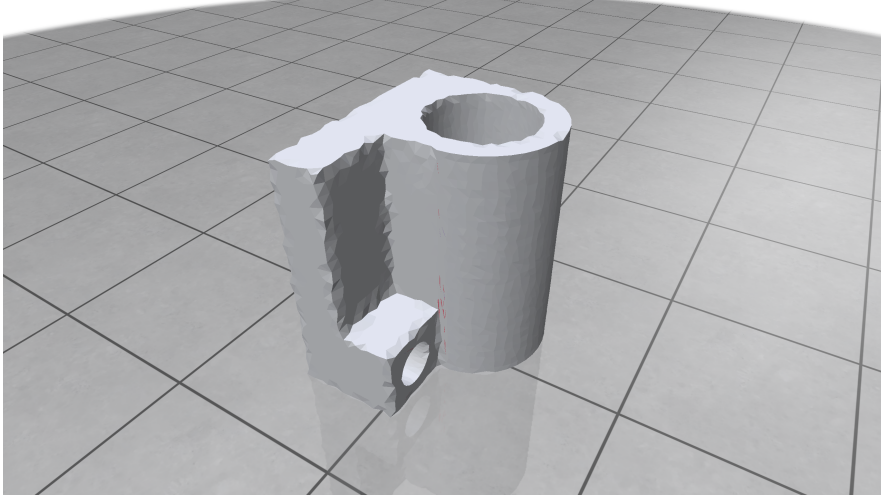


Figure 2: Conservative uniform remeshing result. The number of bad triangles is greatly reduced, but some red and purple triangles remain.

5.3 Targeted Degenerate Cleanup

Uniform remeshing alone does not remove all degenerate triangles. The next stage uses the distinction between needles and caps:

- for needle-like triangles, collapse the shortest edge;
- for cap-like triangles, split the longest edge.

Each candidate operation is checked before it is accepted. Operations that would introduce non-manifold edges or boundary changes on closed meshes are rejected. This targeted cleanup stage reduced the remaining bad triangles on the joint model from 43 to 0.

5.4 Adaptive Remeshing

The adaptive stage follows the idea of Donyach et al. by replacing a single global target edge length with a local sizing field. The implementation uses a simple curvature proxy based on

vertex normal variation over one-ring edges. The resulting rule is:

- high normal variation: smaller target edge length;
- low normal variation: larger target edge length.

On the joint model, adaptive remeshing improved the average minimum angle and reduced the number of triangles compared to the uniform-cleanup result. A final targeted cleanup pass was then applied to remove the few bad triangles created by adaptive operations.

To test whether the visible deformation came mainly from uniform remeshing, a no-uniform experiment was also evaluated. In this variant, adaptive remeshing is applied directly to the original joint input mesh, followed by a targeted cleanup pass. This variant preserves the original appearance better in visual comparison, but it does not reach zero bad triangles. The best no-uniform candidate reduced the joint model from 258 bad triangles to 16.

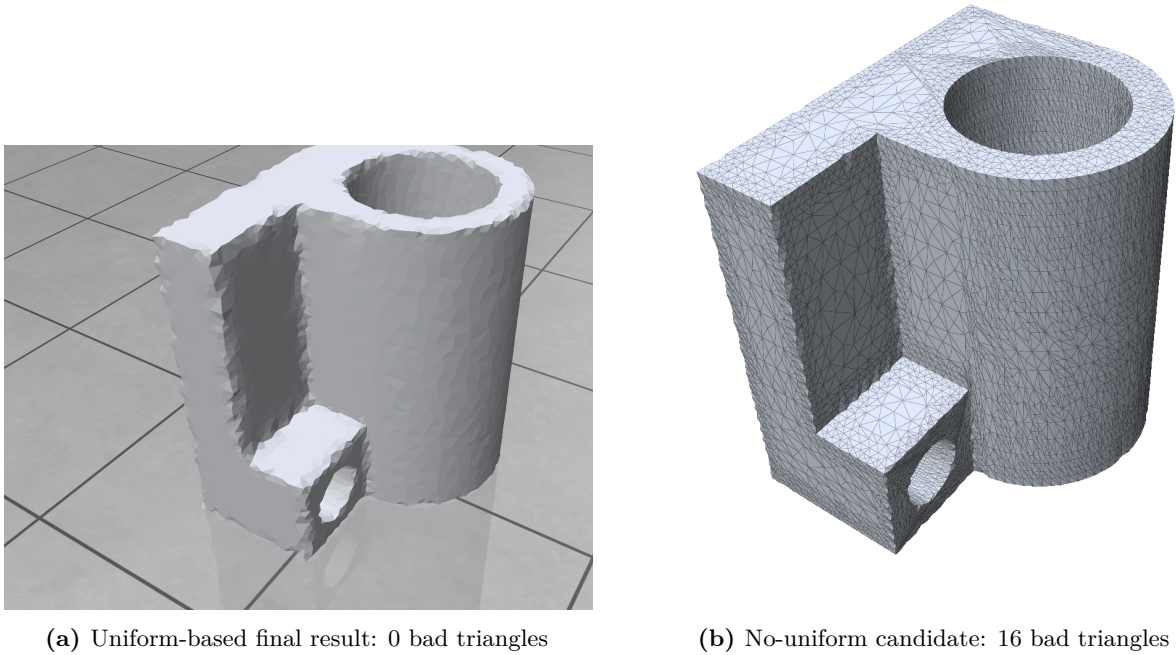


Figure 3: Joint model trade-off. The uniform-based pipeline is stronger according to the bad-triangle metric, but the no-uniform pipeline better preserves the original shape. Reporting both outputs makes the quality–fidelity trade-off explicit instead of treating the zero-bad-triangle result as the only meaningful result.

6 Results

6.1 Joint Model

Table 3: Joint model results. Angles are in degrees. The no-uniform candidate is included because it better explains the shape-preservation trade-off.

Mesh	V	F	Min angle	Avg min angle	Bad triangles	Boundary	Non-manifold
joint_input.off	221	446	0.48	9.35	258	0	0
joint_uniform_safe.off	6537	13078	0.06	35.19	43	0	0
joint_adaptive_final.off	6061	12126	5.65	42.13	0	0	0
joint_input_direct_adaptive_cleanup_i8.off	11996	23996	0.71	23.78	16	0	0
joint_output.off	3400	6804	36.00	53.00	0	0	0

The final adaptive result reaches the same bad-triangle count as the provided reference output: zero. However, this result is built on top of the uniform remeshing stage, and visual inspection shows that this stage can deform the original joint shape. The no-uniform candidate does not remove every bad triangle, but it is useful because it separates shape preservation from pure triangle-quality optimization. In other words, the experiments show a real trade-off: the uniform-based pipeline is better according to the current bad-triangle metric, while the no-uniform candidate is more conservative with respect to the original shape.

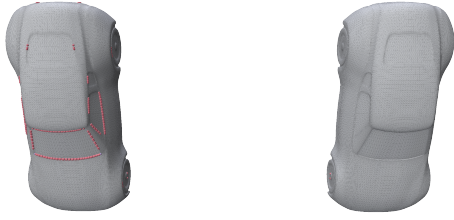
6.2 Car Models

The car models are already much denser than the joint model. For these meshes, targeted cleanup was more practical than full adaptive remeshing. Direct adaptive remeshing on **car4** over-refined the mesh and became computationally expensive, so it was not used as the final car-model pipeline. The car experiments therefore use the same targeted cleanup rule repeatedly: collapse safe shortest edges for needle-like triangles and split safe longest edges for cap-like triangles.

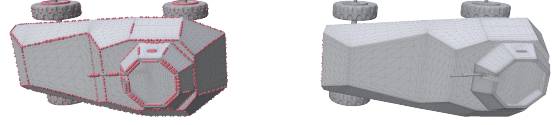
Table 4: Car model results. Angles are in degrees.

Mesh	V	F	Min angle	Avg min angle	Bad triangles	Boundary	Non-manifold
car1.off	48798	97536	0.73	27.04	1400	64	0
car1_targeted_cleanup.off	48349	96638	4.64	27.28	4	64	0
car2.off	78708	157424	0.22	23.27	11100	0	0
car2_targeted_cleanup.off	73912	147832	0.79	24.70	85	0	0
car3.off	93450	186896	0.49	21.93	23495	0	0
car3_targeted_cleanup.off	84705	169406	0.14	24.16	162	0	0
car4.off	14317	28630	0.59	18.76	4787	0	0
car4_targeted_cleanup.off	12308	24612	0.95	21.68	5	0	0

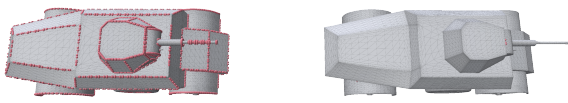
For **car1**, the original mesh already has 64 boundary edges. The cleanup stage preserves this count and does not introduce non-manifold edges. The other car models start closed and remain closed. In ordinary shaded views, the car-model differences are often visually subtle because the meshes are dense and the problematic triangles are local. Therefore, quality-highlight views are used to make the affected regions easier to inspect. The result on **car3** is especially interesting: the total number of bad triangles drops strongly, but a small number of cap-like and needle-like cases remain because the cleanup rejects operations that would be unsafe or would create new local degeneracies.



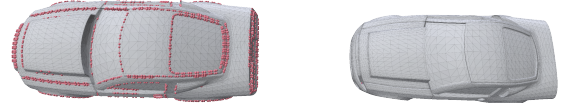
(a) car1: top quality view



(b) car2: top quality view



(c) car3: top quality view



(d) car4: top quality view

Figure 4: Quality-highlight views for all car models. These views are more informative than plain shaded screenshots because the remaining bad triangles are local and can otherwise be hard to notice by eye.

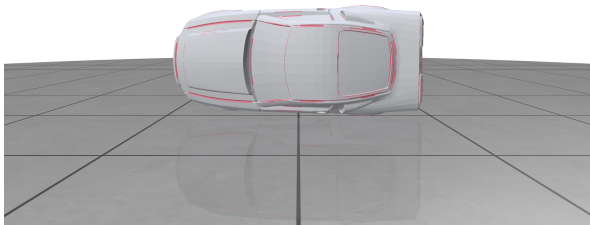


(a) car2

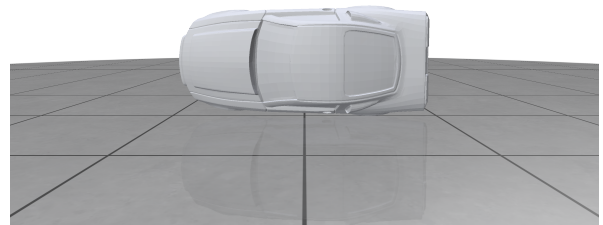
(b) car3

(c) car4

Figure 5: Additional side views for the car models where local defects were easier to inspect from the side. These figures support the numerical table by showing where the cleanup affected local triangle quality.



(a) car4.off



(b) car4_cleanup.off

Figure 6: Car4 before and after targeted cleanup. The bad triangle count is reduced from 4787 to 5.

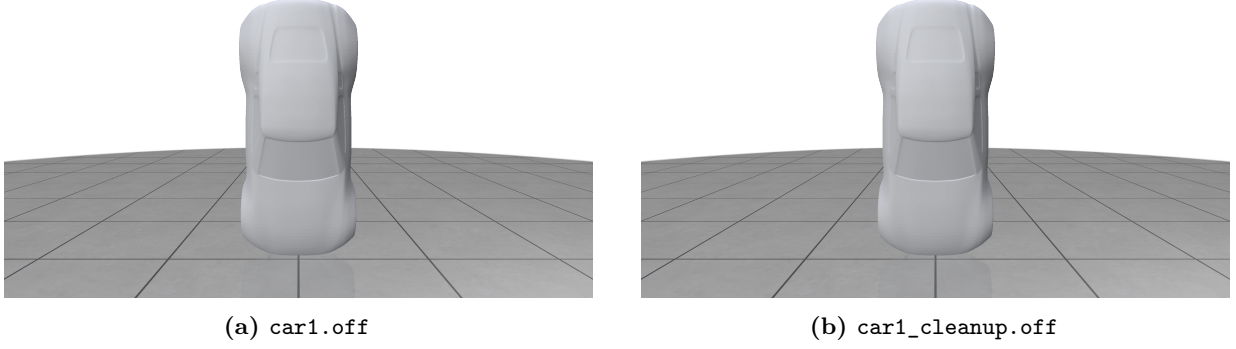


Figure 7: Car1 before and after targeted cleanup. The input already has boundary edges, and the cleanup preserves the boundary count.

6.3 Car3 Local Case Study

To make the local behavior of the cleanup algorithm easier to follow, several representative bad-triangle regions from `car3.off` were examined in detail. A direct face-index tracking approach is not reliable here, because edge collapses and edge splits delete some triangles and create new ones. Instead, the analysis tracks local regions. For each selected bad triangle in the original `car3.off`, its centroid is stored, and at every cleanup step the nearest triangle to that centroid is found. A local patch around the same seed point is also extracted. This makes the tracking stable even when the actual face index changes.

Figure 8 shows four representative regions. Region 1 starts as a cap-like triangle. Regions 2–4 start as needle-like triangles. Yellow marks the tracked local face, red marks needle-like triangles, purple marks cap-like triangles, and gray marks good triangles.

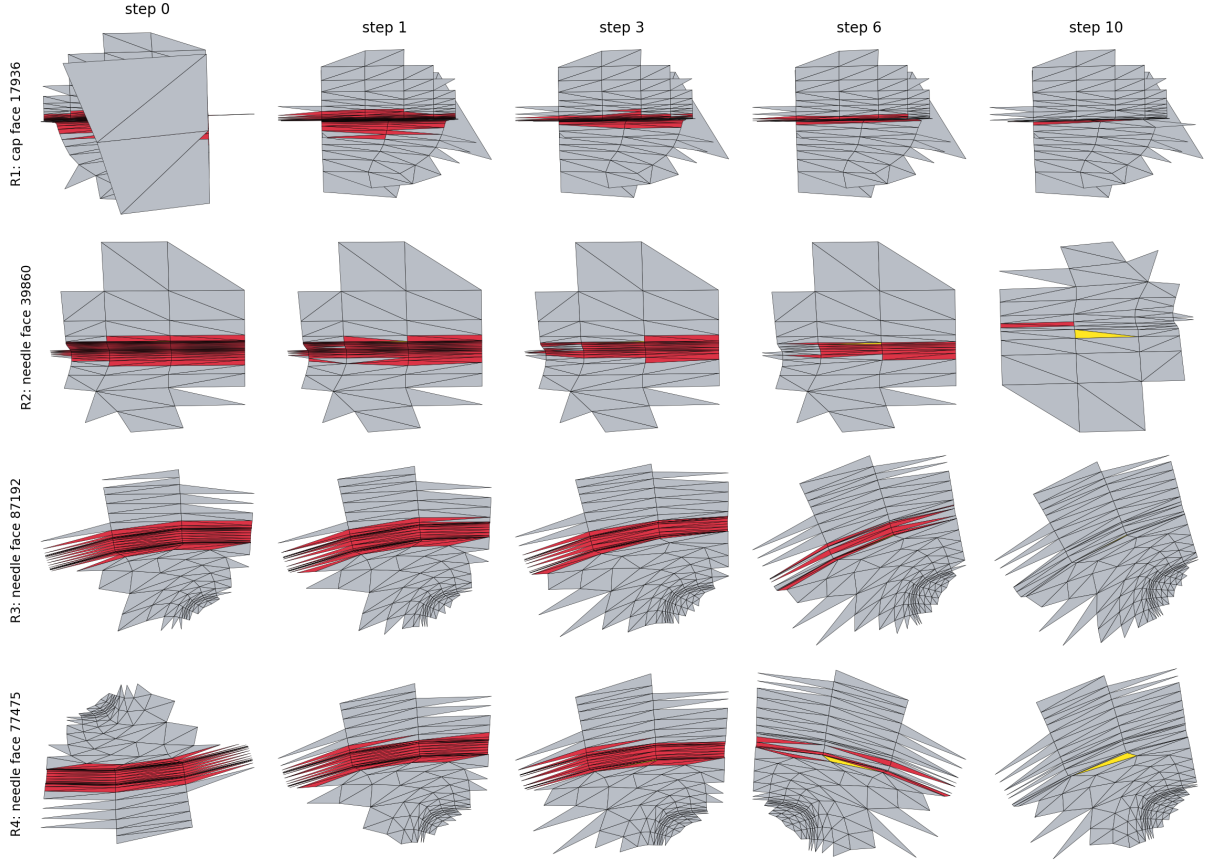


Figure 8: Car3 local diagnostic examples. Each row tracks one bad region through cleanup steps 0, 1, 3, 6, and 10. The tracked local face becomes good in all four cases, while the surrounding patch also loses most of its bad triangles.

Table 5: Tracked car3 regions. The tracked local face is evaluated at the seed point in step 0 and again after step 10.

Region	Seed type	Seed face	Tracked label	Patch bad count	Tracked min angle
R1	cap	17936	cap $\rightarrow$ good	141 $\rightarrow$ 12	1.08 $\rightarrow$ 14.59
R2	needle	39860	needle $\rightarrow$ good	122 $\rightarrow$ 2	0.57 $\rightarrow$ 8.52
R3	needle	87192	needle $\rightarrow$ good	105 $\rightarrow$ 0	0.79 $\rightarrow$ 5.62
R4	needle	77475	needle $\rightarrow$ good	105 $\rightarrow$ 0	0.85 $\rightarrow$ 6.34

The global car3 cleanup is shown in Table 6. The bad triangle count decreases from 23495 to 162 while preserving closed topology. The remaining bad triangles are not ignored; they are cases where the conservative operation checks prevent further changes or where later splits introduce new cap-like cases. This behavior is useful to report because it shows the algorithm’s stopping limitation rather than hiding it.

Table 6: Car3 targeted cleanup progress.

Step	V	F	Bad triangles	Collapses	Splits
0	93450	186896	23495	0	0
1	91406	182808	18500	2057	13
3	88141	176278	10331	1427	18
6	85483	170962	2632	660	8
10	84705	169406	162	46	31

7 Encountered Problems and Observations

Several implementation issues were useful for understanding the problem:

1. **Naive splitting can break topology.** The first remeshing attempt split individual triangles independently. This created missing faces and visible cracks. The fix was globally consistent edge splitting.
2. **Improving triangle angles is not enough.** Some early outputs had better angle statistics but worse topology. This is why boundary and non-manifold edge counts are reported in all experiments.
3. **Smoothing may deform the shape.** Laplacian-like smoothing improved triangle regularity but visibly changed the object. Projection back to the original surface helped, but the final conservative pipeline avoids relying on smoothing.
4. **Uniform remeshing has a shape-preservation cost.** Even without smoothing, global uniform remeshing can move the result away from the original visual shape because it resamples the entire object. The no-uniform joint experiment reduced this problem visually, but it left 16 bad triangles instead of reaching zero.
5. **Cleanup should be type-specific.** Splitting the longest edge of every bad triangle made some cases worse. The better rule was to collapse shortest edges for needles and split longest edges for caps.
6. **Adaptive remeshing is useful but sensitive.** It improved the joint result, but on larger car models it over-refined the mesh without additional acceleration or better parameter control.
7. **Feature preservation remains important.** Some small bumps near sharp edges remain visible. This is expected because explicit feature-edge preservation using dihedral angles is not implemented.

8 Limitations

This project deliberately keeps the method simple and transparent. The main limitations are:

- no explicit feature-edge preservation,
- no exact Hausdorff-distance error bound,
- no spatial acceleration for nearest-surface or adaptive operations,

- cleanup stopping criteria are still parameter-based,
- adaptive remeshing parameters are tuned manually.

These limitations are acceptable for the scope of the term project, but they also suggest clear future improvements.

9 Conclusion

The project produced a working and measurable mesh-quality enhancement pipeline. The implemented method successfully removes all detected bad triangles from the provided joint input mesh while preserving topology. It also performs well on additional car models, reducing thousands of detected bad triangles to only a few remaining cases without introducing non-manifold edges.

The most important lesson from the project is that robust mesh processing should not be evaluated only by visual appearance or triangle-angle statistics. Topology preservation, failure cases, and parameter sensitivity must also be measured. The final pipeline reflects this lesson: it combines quality analysis, visualization, conservative operations, targeted cleanup, and adaptive remeshing into a reproducible workflow.

10 Reproducibility

The main commands are documented in `README.md`. The code is organized under the `meshfix/` package and can be run from the project root using:

```
python3 -m meshfix.cli
```

Important output files include:

- `outputs/meshes/joint_adaptive_final.off`
- `outputs/meshes/joint_input_direct_adaptive_cleanup_i8.off`
- `outputs/meshes/car1_targeted_cleanup.off`
- `outputs/meshes/car2_targeted_cleanup.off`
- `outputs/meshes/car3_targeted_cleanup.off`
- `outputs/meshes/car4_targeted_cleanup.off`
- `outputs/experiment_summary.csv`
- `outputs/joint_adaptive_final_metrics.csv`
- `outputs/joint_no_uniform_experiment_metrics.csv`
- `outputs/car3_diagnostic/car3_region_tracking.csv`

The car3 diagnostic figures can be regenerated with:

```
python3 scripts/car3_diagnostic.py --out-dir outputs/car3_diagnostic_v3
python3 scripts/render_car3_diagnostic_figures.py \
  --out-dir outputs/car3_diagnostic_v3
```

References

- [1] Mario Botsch and Leif P. Kobbelt. A robust procedure to eliminate degenerate faces from triangle meshes. In *Vision, Modeling, and Visualization*, 2001.
- [2] Marion Donyach, David Vanderhaeghe, Loic Barthe, and Mario Botsch. Adaptive remeshing for real-time mesh deformation. In *Eurographics Short Papers*, 2013.